Course Outlines

COMP 3415 – Software Engineering



Department of Computer Science, Lakehead University
955 Oliver Rd, Thunder Bay, ON, P7B 5E1, Canada

Contents

1 Course Information **2**
- 1.1 Catalog Description . . . 2
- 1.2 Course Prerequisite(s) and Logistics . . . 2
- 1.3 Course Instructor . . . 2
- 1.4 Teaching Assistants . . . 2
- 1.5 Learning Outcomes . . . 3
- 1.6 Course Overview . . . 3

2 Course Topics and Details **6**

3 Recommended Readings/Textbooks **7**

4 Assessment and Grading **8**
- 4.1 Evaluation Components . . . 8
- 4.2 Grading Scheme . . . 9
  - 4.2.1 Assignments . . . 9
  - 4.2.2 Quizzes . . . 9
  - 4.2.3 Group Software Engineering Project . . . 10
  - 4.2.4 Assessment Philosophy . . . 13
  - 4.2.5 Assessment-at-a-Glance . . . 14

5 Course Policies **14**
- 5.1 Submission of Course Work . . . 14
- 5.2 Late and Missed Submissions . . . 15
- 5.3 Attendance and Participation . . . 15
- 5.4 Course Communication . . . 15
- 5.5 Course Schedule and Changes . . . 15
- 5.6 Use of Generative AI . . . 15
- 5.7 Academic Integrity . . . 16
- 5.8 Group Project Responsibilities . . . 17
- 5.9 Recording of Lectures and Course Activities . . . 17
- 5.10 Copyright and Course Materials . . . 17
- 5.11 Student Accessibility Services . . . 17
- 5.12 Student Support Resources . . . 18
- 5.13 University Regulations . . . 18
- 5.14 Course Completion . . . 18

6 Land Acknowledgment **18**

1 Course Information

1.1 Catalog Description

An introduction to a spectrum of software engineering concepts: software requirements and specification, project planning and design, software development and testing, software process management, and software maintenance. Several of these topics of software design and testing will be explored in detail. Object-oriented methodology and UML will be used. A team project and class discussion will be an integral part of the course.

1.2 Course Prerequisite(s) and Logistics

Prerequisite(s): COMP 2412 Data Structures and COMP 2477 Object-Oriented Programming

Lecture: Mon/Wed, 1:00–2:30 PM, BB 1054
Lab: Tuesday, 10:30–11:30 AM, AT 4019
Course website: MyCourseLink (D2L)

1.3 Course Instructor

Saikat Mondal, PhD
Office: AT 4006
Office hours: By email/appointment
Email: saikat.mondal@lakeheadu.ca

1.4 Teaching Assistants

Connor McElroy
Qin Zhao

1.5 Learning Outcomes

By the end of this course, students are expected to:

1. understand fundamental software engineering principles, software processes, and the software development lifecycle, and distinguish software engineering from programming and other computing disciplines.
2. compare and apply software development processes, including plan-driven and Agile approaches, Extreme Programming (XP), and Scrum, and develop user stories and acceptance criteria for software requirements.
3. plan and manage software development activities using effort estimation, risk management, iteration and sprint planning, and collaborative development practices.
4. design and implement effective software testing strategies using unit and integration testing, Test-Driven Development (TDD), Continuous Integration (CI), version control, and automated software builds.
5. identify software quality and maintainability problems, including code smells and technical debt, and apply refactoring, clean code, SOLID principles, modularity, coupling, and cohesion to improve software quality.
6. model and design software systems using UML and appropriate design patterns, and make informed architectural decisions based on system requirements and quality attributes.
7. understand and evaluate modern software architectures, including distributed systems, services and APIs, service-oriented architecture, microservices, and cloud-native software concepts.
8. analyze software maintenance and evolution challenges, including program comprehension, debugging, legacy systems, reengineering, refactoring, and technical debt, including challenges associated with maintaining AI-generated software.
9. effectively use and critically evaluate AI-assisted software engineering approaches for requirements, design, code generation and comprehension, testing, debugging, code review, refactoring, and documentation.
10. understand and apply agentic software engineering workflows involving planning, tool use, repository-level development, testing, and review, while maintaining appropriate human oversight and evaluating the risks and limitations of autonomous software engineering agents.
11. demonstrate professional and ethical software engineering practices, including responsible use of AI, attribution, accountability, privacy, and human oversight, and critically assess emerging trends and the evolving role of software engineers.

1.6 Course Overview

This course provides a comprehensive introduction to the principles, processes, practices, and emerging trends of software engineering. It combines foundational software engineering concepts with practical development practices and contemporary approaches, including AI-assisted and agentic software engineering. Students will develop the knowledge and skills

needed to plan, design, develop, test, maintain, and evolve high-quality software systems while considering professional, ethical, and human-centered aspects. The major topics covered include:

❶ **Software Engineering Foundations and Development Processes.** This section introduces the fundamental concepts of software engineering, including software processes, the software development lifecycle, and the characteristics of high-quality software. Students will explore major software process models and compare plan-driven and Agile development approaches. Agile principles and practices, including Extreme Programming (XP), Scrum, user stories, and acceptance criteria, will be introduced to establish a foundation for modern iterative and collaborative software development.

❷ **Software Project Planning and Management.** Effective planning and management are essential for successful software development. This section covers the planning game, effort estimation, risk management, iteration and sprint planning, and collaborative development practices such as pair programming. Students will learn how software teams estimate, prioritize, organize, and manage development activities while responding to changing requirements and project risks.

❸ **Software Testing and Continuous Integration.** This section focuses on systematic approaches to verifying and validating software. Students will learn about unit, integration, system, and acceptance testing, with particular emphasis on Test-Driven Development (TDD). Continuous Integration (CI), automated software builds, version control, and documentation practices will also be explored to demonstrate how testing and automation can be integrated throughout the software development process.

❹ **Software Quality, Clean Code, and Refactoring.** Software quality and maintainability are central concerns in long-lived software systems. Students will examine software quality, technical debt, common code smells, and principles and techniques for systematic refactoring. The section also introduces clean code and SOLID design principles, modularity, separation of concerns, coupling, and cohesion to help students develop software that is understandable, testable, maintainable, and adaptable to change.

❺ **Software Design, Modeling, and Design Patterns.** This section introduces software modeling and design techniques for representing and communicating software structure and behavior. Students will work with selected UML models, including use-case, class, and sequence diagrams. The section also introduces software design patterns and demonstrates how established patterns can be applied to recurring design problems to improve modularity, flexibility, and maintainability.

❻ **Software Architecture and Modern Software Systems.** Students will explore the principles of software architectural design, architectural views, common architectural styles and patterns, components, frameworks, and architectural trade-offs. The course will extend these foundations to modern software architectures, including distributed systems, services and APIs, service-oriented architecture, microservices, and cloud-native software concepts. Emphasis will be placed on understanding how architectural decisions affect quality attributes such as scalability, maintainability, reliability, and performance.

❼ **Software Maintenance and Evolution.** Software systems must continuously evolve in response to changing requirements, technologies, and environments. This section covers

software evolution and maintenance, program comprehension, debugging, legacy systems, software reengineering, refactoring, and technical debt. Students will also examine emerging challenges associated with understanding, evolving, and maintaining software produced with AI-assisted and agentic development tools.

8 **AI-Assisted Software Engineering.** Generative AI and Large Language Models (LLMs) are increasingly transforming software development practices. This section examines AI-assisted activities across the software lifecycle, including requirements and design, code generation and comprehension, testing, debugging, code review, refactoring, and documentation. Students will learn to critically evaluate AI-generated software with respect to correctness, maintainability, security, and quality, while also examining limitations such as hallucinations and the responsible use of AI in software development.

9 **Agentic Software Engineering.** This section extends AI-assisted development toward autonomous and semi-autonomous software engineering agents. Students will explore agentic development workflows involving planning, tool use, repository-level code modification, implementation, testing, debugging, and review. Topics include specification-driven development, human-in-the-loop approaches, multi-agent workflows, and appropriate boundaries between human and agent responsibilities. The risks and limitations associated with increasingly autonomous software development will also be examined.

10 **Professional Software Engineering and Future Trends.** This section examines software engineering as a professional discipline, emphasizing user-centered development, professional responsibility, ethics, accountability, and human oversight. Particular attention will be given to responsible AI-assisted software development, including attribution, provenance, privacy, intellectual property, and accountability for AI-generated software. The course concludes by examining emerging software engineering practices and the evolving role of software engineers in increasingly AI-assisted and agentic development environments.

2 Course Topics and Details

Week	Date	Topic	Reading / Reference	Assessment
W1	2026-09-09	**Introduction to Software Engineering:** Course overview and objectives. What is Software Engineering? Software processes, software development lifecycle, characteristics of good software, and modern software development.	Sommerville, Ch. 1	—
W2	2026-09-14 2026-09-16	**Software Process Models and Agile Development:** Software process models, plan-driven vs. Agile development, Agile principles, Extreme Programming (XP), Scrum, user stories, and acceptance criteria/tests.	Sommerville, Chs. 2–3; selected Ch. 4	P0
W3	2026-09-21 2026-09-23	**Software Project Planning and Management:** Planning game, risk management, effort estimation, iteration/sprint planning, and pair programming.	Sommerville, Ch. 3; selected Project Management chapters	—
W4	2026-09-28 2026-09-30	**Software Design and Modeling:** UML, use-case, class and sequence diagrams, software design principles, design patterns, and their applications.	Sommerville, Chs. 5 & 7; Gamma et al., *Design Patterns*	P1, A1
W5	2026-10-05 2026-10-07	**Software Architecture:** Architectural design, architectural views and patterns, layered and client-server architectures, components and frameworks, and architectural trade-offs.	Sommerville, Ch. 6	Q1
—	—	**Reading Week (Fall Study Break)**	—	—
W6	2026-10-19 2026-10-21	**Modern Software Architectures:** Distributed software systems, services and APIs, service-oriented architecture, microservices, and cloud-native software concepts.	Selected Sommerville distributed systems chapters; supplementary material	P2
W7	2026-10-26 2026-10-28	**Clean Code and Software Design Principles:** Clean code, SOLID principles, modularity, coupling and cohesion, separation of concerns, and maintainability.	Martin, *Clean Code*; supplementary material	—
W8	2026-11-02 2026-11-04	**Software Testing and Continuous Integration:** Software testing, unit and integration testing, Test-Driven Development (TDD), Continuous Integration (CI), software builds, version control, and documentation.	Sommerville, Ch. 8	P3, A2
W9	2026-11-09 2026-11-11	**Software Quality, Code Smells, and Refactoring:** Software quality, technical debt, code smells, refactoring principles and techniques, and maintaining code quality.	Fowler, *Refactoring*; selected Sommerville material	Q2
W10	2026-11-16 2026-11-18	**Software Maintenance and Evolution:** Software evolution and maintenance, program comprehension, debugging, legacy systems, reengineering, refactoring, technical debt, and maintenance of AI-generated software.	Sommerville, Ch. 9	P4

Week	Date	Topic	Reading / Reference	Assessment
W11	2026-11-23 2026-11-25	**AI-Assisted Software Engineering:** LLM-based coding assistants, AI-assisted requirements/design, code generation and comprehension, testing, debugging, code review and refactoring, evaluation of AI-generated code, limitations, and responsible use.	Recent literature and supplementary material	—
W12	2026-11-30 2026-12-02	**Agentic Software Engineering:** Coding agents, agentic development workflows, planning and tool use, repository-level agents, specification-driven development, implementation–testing–review loops, human-in-the-loop and multi-agent workflows, risks and limitations.	Recent literature and supplementary material	P5, A3
W13	2026-12-07	**Professional Software Engineering and Future Trends:** User-centered development, professionalism and ethics, responsible AI-assisted development, attribution and accountability, human oversight, emerging software engineering practices, future trends, and course recap.	Selected Sommerville material; supplementary material	P6, Q3

Note: The instructor may adjust selected weekly topics, as appropriate, and may replace certain topics with relevant guest lectures (**A** = Assignment; **Q** = Quiz; **P** = Project Milestones).

3 Recommended Readings/Textbooks

There is no required textbook for the course. However, a software engineering textbook is highly recommended as an essential resource for understanding the fundamental concepts covered in the course. The recommended textbook is:

- Ian Sommerville, **Software Engineering**, 10th Edition or later, Pearson/Addison-Wesley.

The tentative class schedule identifies the relevant chapters from this textbook where appropriate. Additional readings, lecture materials, and recent research articles will be provided for selected topics, particularly modern software architectures, AI-assisted software engineering, agentic software engineering, and emerging trends in software development.

Additional Resources

[1] Kenneth S. Rubin, **Essential Scrum: A Practical Guide to the Most Popular Agile Process**. Addison-Wesley, 2012.

[2] Martin Fowler, **Refactoring: Improving the Design of Existing Code**. 2nd Edition, Addison-Wesley, 2018.

[3] Robert C. Martin, **Clean Code: A Handbook of Agile Software Craftsmanship**. Prentice Hall, 2008.

[4] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, **Design Patterns: Elements of Reusable Object-Oriented Software**. Addison-Wesley, 1994.

[5] Selected recent research papers, technical articles, and supplementary materials on AI-assisted software engineering, agentic software engineering, and contemporary software development practices will be provided through the course website.

4 Assessment and Grading

The course assessments are designed to evaluate both **individual understanding** and the ability to **apply software engineering practices in realistic development settings**. Rather than assessing only the final software product, the course emphasizes engineering reasoning, evidence, testing, design decisions, adaptation to change, and responsible use of AI.

The assessment progresses through three complementary stages:

Understand and Decide $\rightarrow$ **Test and Improve** $\rightarrow$ **Design and Evaluate** $\rightarrow$ **Build, Change, and Evolve**

Individual assignments establish core software engineering competencies. Quizzes assess conceptual understanding and the ability to apply course concepts to short scenarios. The group project integrates the complete software engineering process and introduces unexpected requirement changes followed by AI-assisted software evolution.

4.1 Evaluation Components

The course workload consists of the following components:

- **Assignments (35%):** Three individual assignments assess complementary software engineering competencies. Assignment 1 focuses on requirements, process selection, planning, estimation, and risk. Assignment 2 focuses on testing, software quality, code smells, refactoring, and responding to a software change. Assignment 3 focuses on software modeling, design patterns, architecture, architectural trade-offs, and the critical evaluation of AI-generated recommendations.
- **Quizzes (20%):** Three individual quizzes will assess understanding of the course concepts and their application to short software engineering scenarios. The quizzes are intended to encourage continuous learning rather than end-of-term memorization.
- **Group Software Engineering Project (45%):** Students will work in teams to develop and evolve a small but complete software system. The project follows an incremental software engineering process involving requirements, planning, architecture, implementation, testing, change impact analysis, and final delivery. After establishing a conventional engineering baseline, teams will respond to an instructor-provided requirement change and later complete a second change using AI-assisted development. The emphasis is on engineering quality, reasoning, verification, and the ability to adapt an existing software system.

Table 2: Grading Scheme

Assessment	Tentative Timing	Weight
Assignment 1: Software Engineering Decision Challenge	Around Week 4	10%
Assignment 2: Software Rescue Mission	Around Week 8	12%
Assignment 3: Architecture Court – You vs. AI	Around Week 12	13%
Quiz 1	Around Week 5; Topics from Weeks 1–4	6%
Quiz 2	Around Week 9; Topics from Weeks 5–8	6%
Quiz 3	Around Week 13; Topics from Weeks 9–12	8%
Group Project	Throughout the term	45%
Final Exam	None	0%

4.2 Grading Scheme

4.2.1 Assignments

The three assignments are intentionally designed to assess different aspects of software engineering.

- **Assignment 1 – Software Engineering Decision Challenge (10%):** Students analyze an instructor-provided software project scenario containing incomplete and ambiguous requirements. They refine requirements, develop user stories and acceptance criteria, select and justify a development process, estimate and prioritize work, plan iterations, analyze risks, and respond to an unexpected requirement change. The emphasis is on **case-specific engineering reasoning rather than memorizing definitions**. Generative AI is not permitted for this assignment.
- **Assignment 2 – Software Rescue Mission (12%):** Students inherit a small instructor-provided software system with limited tests, code-quality problems, and one or more defects. They first understand and test the existing software, identify significant engineering problems, and then apply appropriate refactoring techniques. A new client requirement is subsequently introduced, allowing students to evaluate whether their earlier design improvements made the software easier to change. After an initial human-only checkpoint, limited AI assistance may be used and must be critically evaluated.
- **Assignment 3 – Architecture Court: You vs. AI (13%):** Students independently design and model a software system using UML, design patterns, and architectural principles. After submitting their initial design, they use an AI system to critique the architecture. Students must classify selected AI recommendations as **Accept, Modify, or Reject** and justify each decision using requirements, quality attributes, architectural trade-offs, and project constraints. An additional architecture change is then introduced to assess whether students can adapt their design when important assumptions change.

4.2.2 Quizzes

Three individual quizzes will be conducted during the term.

The quizzes will contain a combination of:

- conceptual questions;
- short scenario-based questions;

- interpretation of software engineering artifacts; and
- small design, testing, process, or architecture decisions.

The emphasis will be on **understanding and application**. Questions that require only memorization of terminology will be kept to a minimum.

Quiz	Primary Coverage	Weight
Quiz 1	Software engineering foundations, process models, Agile/XP/Scrum, planning, risk, testing, TDD, and CI	6%
Quiz 2	Software quality, code smells, refactoring, clean code, SOLID, software modeling, design patterns, and architecture	6%
Quiz 3	Modern architectures, maintenance/evolution, AI-assisted software engineering, agentic software engineering, professionalism, and responsible AI	8%

4.2.3 Group Software Engineering Project

Students will work in groups of approximately four students to design, implement, test, and deliver an instructor-approved software system using an **incremental Agile/Scrum-inspired development process**.

The purpose of the project is not to build the largest possible application. Instead, the emphasis is on experiencing a realistic software engineering workflow in which teams define and prioritize requirements, plan iterations, develop working software incrementally, continuously test and integrate their work, review progress, update the backlog, and deliver a stable final system.

The overall project progression is:

Requirements and Backlog $\rightarrow$ **Plan** $\rightarrow$ **Build and Test** $\rightarrow$ **Review** $\rightarrow$ **Adapt** $\rightarrow$ **Next Iteration** $\rightarrow$ **Final Release**

Projects should be appropriately scoped and should avoid unnecessary dependence on paid services, specialized hardware, large external datasets, or difficult-to-access third-party resources.

Throughout the project, teams are expected to maintain an evolving **product backlog**, use version control, distribute work among team members, test completed functionality, and reflect briefly on each development iteration. Project artifacts should evolve throughout the semester rather than being recreated for every milestone.

Project Milestones

P0 – Team Formation and Project Selection: Students will form a group of approximately four students and select an instructor-approved project. If required, the instructor may assist with group formation. Teams will establish a shared repository, identify the primary users of the system, select an appropriate technology stack, and define an appropriately scoped software problem.

P1 – Requirements and Project Plan: Teams will establish the initial project baseline by defining stakeholders, functional and non-functional requirements, and a prioritized product backlog.

Major functionality should be represented through user stories containing a clear description, expected deliverable, and testable acceptance criteria. Teams will estimate the relative effort of their user stories, prioritize the backlog, identify important project risks and assumptions, and prepare an initial iteration plan.

Table 3: Group Software Engineering Project Milestones

Milestone	Primary Deliverables	Weight
P0: Team Formation and Project Selection	Team formation, project description, user roles, technology selection, and repository setup.	Mandatory
P1: Requirements and Project Plan	Requirements, user stories (description, deliverables, and acceptance criteria), user story estimation and prioritization, product backlog, initial iteration plan, project risks, and important assumptions.	7%
P2: Engineering Baseline	Initial working functionality (e.g., login/homepage or another core workflow), architecture/design artifact, testing plan and initial automated tests, version-control evidence, updated backlog, and a short iteration review.	10%
P3: Iteration Release I	Target approximately 50% of the planned project functionality completed and tested, updated backlog and iteration status, working software increment, and a short iteration review.	5%
P4: Iteration Release II	Target approximately 75% of the planned project functionality completed and tested, updated backlog and iteration status, working software increment, and a short iteration review.	5%
P5: Final Engineering Release	Completed and stable implementation, final testing and bug fixing, updated architecture/design and backlog, documentation, and a concise final engineering report.	15%
P6: Final Demo and Engineering Defense	Final system demonstration, requirements-to-implementation-to-test traceability, major engineering decisions, development process and iteration experience, team contributions, and individual questions.	3%

The deliverable is intentionally concise. The objective is to create **working engineering artifacts that guide development**, rather than a large requirements document that is produced once and not subsequently used.

P2 – Engineering Baseline: Teams will establish the initial working foundation of the software system. This may include basic functionality such as authentication, navigation, persistent data, or another core workflow appropriate to the selected project.

Teams will also establish an initial architecture/design, testing strategy, version-control workflow, and automated tests where appropriate.

The milestone should demonstrate the relationship:

Requirement/User Story $\rightarrow$ **Implementation** $\rightarrow$ **Test/Acceptance Criteria**

Teams will update their product backlog based on actual development progress and submit a short iteration review describing what was planned, what was completed, significant difficulties encountered, and what should be adjusted for the next iteration.

P3 – Iteration Release I: Teams will complete the first substantial development iteration, targeting approximately **50% of the planned project functionality**.

Progress should not be measured only by counting user stories. The complexity, estimated

effort, priority, completion of acceptance criteria, and quality of the implemented functionality will also be considered.

For this milestone, teams should:

- demonstrate a meaningful working software increment;
- complete and test the selected user stories;
- update the product backlog and status of user stories;
- identify incomplete or deferred work;
- update relevant design artifacts where necessary; and
- provide a short iteration review.

The iteration review should briefly identify what was planned, what was completed, what prevented or delayed planned work, and what the team will change in the next iteration.

P4 – Iteration Release II: Teams will continue incremental development, targeting approximately **75% of the planned project functionality**.

This milestone should demonstrate that the team has responded appropriately to the experience and evidence obtained during earlier iterations. Teams may reprioritize, refine, split, or clarify user stories where justified, provided that such decisions are documented in the evolving product backlog.

For this milestone, teams should:

- demonstrate the next stable working software increment;
- complete and test additional prioritized user stories;
- verify previously completed functionality through regression testing;
- update the product backlog and remaining work;
- address significant technical or quality issues identified during earlier iterations; and
- provide a short iteration review.

The objective is not simply to demonstrate more features than P3, but to show that the software system is **evolving systematically through repeated planning, implementation, testing, review, and adaptation**.

P5 – Final Engineering Release: The final milestone focuses on completing and stabilizing the software system.

Teams should complete the agreed project scope, resolve significant defects, execute final testing, and ensure that completed user stories satisfy their acceptance criteria. Relevant architecture/design artifacts and the product backlog should be updated to reflect the final state of the system.

Teams will submit:

- the final working software system;
- final product backlog showing the status of planned user stories;

- final automated/manual testing evidence, as appropriate;
- updated architecture/design documentation;
- installation/build and user documentation;
- final repository; and
- a concise final engineering report.

The final report should summarize the project rather than repeat all previous milestone submissions. It should emphasize the final system, important engineering decisions, testing and quality assurance, how the project evolved across iterations, major challenges, and lessons learned.

P6 – Final Demo and Engineering Defense: Each team will demonstrate the final software system and explain the engineering process used to develop it.

The demonstration and discussion should include:

- demonstration of the major completed functionality;
- one important **requirement/user story** $\rightarrow$ **implementation** $\rightarrow$ **test** trace;
- one significant architecture or design decision and its justification;
- how the product backlog and priorities evolved across iterations;
- one important problem encountered during development and how the team responded to it; and
- a brief reflection on the team's Agile/Scrum-inspired development experience.

Questions may be directed to individual team members to verify their understanding, contribution, and ownership of the submitted work.

The final demonstration is intended as an **engineering defense**, not only a product presentation. Therefore, presentation polish or visual appearance will be secondary to functionality, software quality, testing, traceability, engineering reasoning, and demonstrated understanding.

4.2.4 Assessment Philosophy

Across assignments, quizzes, and project milestones, assessments emphasize five recurring dimensions:

Correctness + Engineering Quality + Reasoning + Process Evidence + Verification

A technically correct solution without adequate justification may receive partial credit. Software engineering decisions should be supported by evidence from requirements, design, implementation, tests, or the development process.

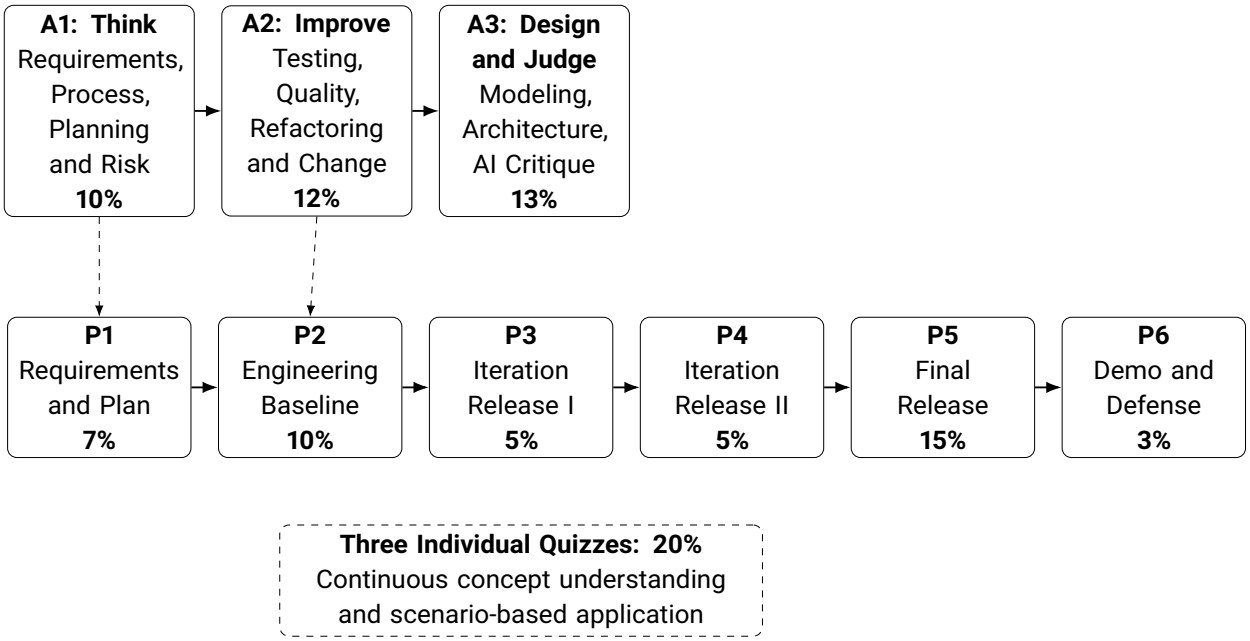
The individual assignments progressively assess students' ability to analyze software engineering problems, improve existing software, make design and architectural decisions, and critically evaluate the use of AI-assisted software engineering.

The group project emphasizes collaborative and iterative software development. Teams are expected to demonstrate how requirements and user stories progress through planning, implementation, testing, review, and adaptation across successive development iterations.

Students remain responsible for all submitted work regardless of whether AI tools are permitted. AI-generated output alone is not evidence of correctness.

4.2.5 Assessment-at-a-Glance

The assessment journey progresses from individual competency development to collaborative and iterative software engineering practice.



The assessment structure can therefore be summarized as:

Think → **Improve** → **Design and Judge** → **Plan** → **Build** → **Iterate** → **Deliver and Defend**

Final Exam: There is no final examination in this course.

5 Course Policies

5.1 Submission of Course Work

Assignments and project deliverables must be submitted through the appropriate submission area on MyCourseLink (D2L), unless otherwise specified by the instructor. For the group software engineering project, teams may be required to maintain their source code, version history, and related development artifacts in a shared GitHub repository. Specific submission requirements, due dates, repository requirements, and additional instructions will be provided for each assessment.

Students are responsible for ensuring that the correct and complete files, links, and other required materials are submitted before the stated deadline. Students should verify their submissions on MyCourseLink and, where applicable, ensure that the submitted project repository is accessible to the instructor and teaching assistants.

5.2 Late and Missed Submissions

Students are expected to submit assignments and project deliverables by the posted deadlines. Late-submission rules and any applicable penalties will be specified with the corresponding assessment.

Students who anticipate being unable to meet a deadline because of medical, compassionate, accessibility, or other significant circumstances should contact the instructor as early as reasonably possible. Extensions or alternative arrangements may be granted when appropriate and in accordance with applicable Lakehead University policies and procedures.

Approved academic accommodations will be respected in accordance with Lakehead University policies and Student Accessibility Services (SAS) procedures.

Failure to submit required course work without an approved extension or accommodation may result in a grade of zero for the missed assessment.

5.3 Attendance and Participation

Regular attendance and active participation are strongly recommended. Lectures may include explanations, examples, discussions, demonstrations, and activities that supplement the posted course materials and readings.

Students are responsible for keeping up with course announcements, materials, assessment requirements, and schedule changes regardless of attendance.

5.4 Course Communication

MyCourseLink (D2L) and Lakehead University email will be the primary channels for course communication. Course announcements, learning materials, assessment information, and schedule updates may be communicated through these channels.

Students are responsible for checking MyCourseLink and their Lakehead University email regularly. Students should use their Lakehead University email account when communicating with the instructor.

5.5 Course Schedule and Changes

The course schedule is tentative and may be adjusted based on the pace and needs of the class, availability of guest speakers, or other academic considerations. Any significant changes to course activities, assessment dates, or requirements will be communicated through MyCourseLink and/or Lakehead University email.

5.6 Use of Generative AI

Generative artificial intelligence (GenAI) tools may be used **only when explicitly permitted by the instructor for a particular assignment, project milestone, or course activity**. Permission to use GenAI for one assessment does not imply permission to use it for other assessments.

Some activities in this course are intentionally designed to assess students' independent software engineering reasoning and therefore may prohibit the use of GenAI. Other activities may

explicitly incorporate AI-assisted software engineering and require students to use, analyze, or critically evaluate AI-generated outputs. The instructions for each assessment will specify the permitted level and purpose of AI use.

When GenAI use is permitted, students remain responsible for all submitted work, including the correctness, quality, security, and appropriateness of AI-generated or AI-assisted content. Students must follow any disclosure, acknowledgement, documentation, or evaluation requirements specified by the instructor.

Unless explicitly permitted, GenAI tools must not be used for quizzes or other assessments designated for independent completion.

Using GenAI tools where prohibited, exceeding the permitted scope of their use, or using them in a manner that misrepresents authorship or individual contribution may constitute academic misconduct under Lakehead University's Student Code of Conduct – Academic Integrity.

Students who are uncertain whether a particular use of AI is permitted must consult the instructor before using the tool.

For Lakehead University's guidance on ChatGPT and other AI tools, see:

ChatGPT & Other AI Tools – Lakehead University

5.7 Academic Integrity

Academic integrity is a fundamental expectation of this course and of Lakehead University. Students are responsible for completing their own original work, appropriately acknowledging sources, collaborating only when authorized, and avoiding misrepresentation.

Unless otherwise specified by the instructor:

- individual assignments must be completed independently;
- quizzes must be completed independently using only authorized resources;
- collaboration on the group project is permitted within the assigned project team;
- external sources, tools, and contributions must be appropriately acknowledged when required; and
- unauthorized assistance, including unauthorized use of generative AI tools, is not permitted.

Plagiarism, cheating, unauthorized collaboration, falsification, unauthorized use of materials or tools, and other forms of academic misconduct will be addressed in accordance with Lakehead University's *Student Code of Conduct – Academic Integrity*.

Students are responsible for familiarizing themselves with the University's academic integrity requirements:

Lakehead University – Student Conduct and Academic Integrity

5.8 Group Project Responsibilities

The software engineering project is collaborative work. Each team member is expected to participate meaningfully and responsibly throughout the project and to understand the major requirements, design decisions, implementation, testing, changes, and other engineering activities associated with the team's work.

Teams may distribute tasks among members, but project work should involve regular communication, coordination, and shared understanding. Submitted project deliverables represent the work of the team.

Questions may be directed to individual team members during project meetings, demonstrations, presentations, or other project evaluations to assess individual understanding and contribution. Significant concerns regarding participation or contribution may be considered when determining individual project performance.

5.9 Recording of Lectures and Course Activities

Students may not record, reproduce, distribute, post, or otherwise share lectures or course activities without permission from the instructor, except where recording is authorized as part of an approved academic accommodation.

Students with an approved recording accommodation through Student Accessibility Services must follow the applicable accommodation requirements and confidentiality provisions. Authorized recordings are for the student's personal educational use and must not be distributed or shared without appropriate permission.

5.10 Copyright and Course Materials

Course materials provided by the instructor, including lecture slides, notes, assignment and project materials, assessment materials, and materials posted to MyCourseLink, are provided for students' personal educational use in this course.

Students must not reproduce, upload, distribute, sell, publish, or otherwise share course materials outside the course without appropriate authorization. Students are expected to respect applicable copyright law, intellectual property rights, and University policies.

5.11 Student Accessibility Services

Lakehead University is committed to providing equitable access to academic activities for students with disabilities and/or medical conditions. Student Accessibility Services (SAS) coordinates academic accommodations for eligible students.

Students who require or believe they may require academic accommodations are encouraged to contact SAS and begin the accommodation process as early as possible. Students with approved accommodations are responsible for following applicable SAS procedures and deadlines.

For information about Student Accessibility Services, see:

Lakehead University – Student Accessibility Services

5.12 Student Support Resources

Lakehead University provides academic, accessibility, health, wellness, and other student-support services. Students are encouraged to access these resources when needed.

- **Student Success Centre:** Academic and learning support services.

  Lakehead University – Student Success Centre

- **Student Accessibility Services (SAS):** Academic accommodations and accessibility support.

  Lakehead University – Student Accessibility Services

- **Student Health and Wellness:** Health and wellness services and resources.

  Lakehead University – Student Health and Wellness

- **Lakehead University Library:** Research resources, databases, learning materials, and research assistance.

  Lakehead University Library

5.13 University Regulations

Students are responsible for familiarizing themselves with and complying with the applicable academic regulations, policies, procedures, and requirements of Lakehead University.

Official academic requirements and regulations are available through the Lakehead University Academic Calendar and University policy resources. Where this course outline and an official University policy or regulation conflict, the applicable University policy or regulation takes precedence.

5.14 Course Completion

This course has **no midterm examination and no final examination**. Final grades are determined from the assignments, quizzes, and group software engineering project according to the grading scheme specified in this course outline.

6 Land Acknowledgment

Lakehead University respectfully acknowledges its campuses are located on the traditional lands of Indigenous Peoples. Lakehead Thunder Bay is located on the traditional lands of the Fort William First Nation, Signatory to the Robinson Superior Treaty of 1850. Lakehead Orillia is located on the traditional territory of the Anishinaabeg. The Anishinaabeg include the Ojibwe, Odawa, and Pottawatomi nations, collectively known as the Three Fires Confederacy. Lakehead University acknowledges the history that many nations hold in the areas around our campuses, and is committed to a relationship with First Nations, Métis, and Inuit Peoples based on the principles of mutual trust, respect, reciprocity, and collaboration in the spirit of reconciliation.